Lab 12: USI Campus Connector

CS 311: Data Structures & Algorithms (Fall 2026)

Points: 100. **Due:** Wednesday, November 18, 2026 at 11:59 PM Central.

Starter repository: GitHub template. Python 3.11+; standard library only.

The Mission

Build a **USI campus connector** that finds the **fewest links** from **Rice Library** to **Orr Center**. Buildings are vertices; fictional two-way connectors are unweighted edges. In a **connector group**, every building links directly to every other member.

Part A: The Proof Detective (50 Points)

Instructions: Answer Questions 1–4 in a separate PDF or Markdown document, using matching numbers and our A–F class methods.

1. **Proof by Induction — BFS Layering:**

Prove by induction on layer `d` that BFS visits every vertex at graph-distance `d` before any at distance `d + 1`. State the **base case** (`d = 0`), assumption and step, and justify **FIFO layer order**. Review slides 23 and 30.

The Brutality Clause: Missing the explicit base case earns **0 on Question 1**, regardless of the inductive step.

2. **Visual Tracing — A Campus Route:**

Use labels **A = Rice Library, B = University Center, C = Liberal Arts Center, D = Science Center, E = Orr Center**. The groups are **Link 1: [A,B], Link 2: [B,D], Link 3: [D,E], Link 4: [A,C]**. Use full building names in code.

- Draw the graph. Start BFS at **A**, state your neighbor order, and record the initial **queue/visited set** and each subsequent state until **E is discovered**.
- Report the minimum number of links. Review slides 11–21 and 28–29.

3. **DFS Discovery and Finish Times — One New Connection:**

Add the **C–D** link. The edges are now `A-B, A-C, B-D, C-D, D-E`.

- Start DFS at **A**, visit labels alphabetically, and record every vertex's **discovery/finish times** with one clock.
- Explain cycle detection using your trace, timestamps or edge classification; distinguish a parent edge from cycle evidence. Review slides 39–53.

4. **PDC Reasoning — Two Workers and One Building:**

Two workers in the **same BFS frontier** check the **same unseen building** before either records discovery. Trace the interleaving and explain the duplicate queue entry or conflicting parent/distance update. Identify **atomic check-and-record operations**, then explain separately why they do not guarantee **BFS layer order**. No threaded code is required. See slides 64–65.

Part B: The Recursive Coder (50 Points)

Instructions: Complete `build_graph`, `has_path` and `bfs_shortest_path` in `run_campus.py`. Clone your own template copy, open its folder in VS Code and run `python run_campus.py`. An untouched starter stops at its first **NotImplementedError**; that is the intended TODO.

Requirements:

1. **Build the graph:** Turn `{connector_group: [building list]}` into an **adjacency list** (`Dict[str, List[str]]`). Connect every pair of buildings in a group, in both directions; groups can contain more than two buildings. Run again and inspect the edge checks before continuing. Review slides 4–8.
2. **Recursive reachability:** Implement `has_path(graph, start, target, visited=None)` using **recursive DFS** as a connectivity sanity check. Handle **start equals target** and an **exhausted search with no path**, and use visited state to terminate on cycles. Review slide 47.
3. **Shortest route:** Implement `bfs_shortest_path` with an explicit **collections.deque queue**. Return the ordered building list forming a shortest route, or **None** when unreachable; return `[start]` for a same-start route. A parent chain is one reconstruction option. Include the required comment explaining why a queue-processing loop is more natural here than recursion. Review slides 24–29 and 35.
4. **Base Case Required:** The recursive helper must terminate correctly for a target match and an exhausted search. A recursive method with no base case receives **0** under the existing base-case policy.
5. **Verification:** Run `python run_campus.py` after completing each phase; a later TODO may stop an intermediate run. Once all **13 supplied checks** pass, enter your own USI username to generate the **Success Token**. The suite checks group expansion, reachability, same-start/disconnected cases and a shortcut. The token does not replace review of recursion, the BFS comment or written reasoning.

Submission

1. **Lab12_Theory.pdf** or **Lab12_Theory.md**, with numbered answers 1–4.
2. **run_campus.py**, containing your completed code.
3. The **Success Token** generated by the script.

Submit through **Blackboard** and include the syllabus **AI Usage note** when AI tools were used. Part A remains 50 points and Part B remains 50 points. The recommended work order and slide references add no new submission item.